

Mantenimiento de Dependencias

Ciclo de vida completo de las dependencias en un monorepo
Flutter + Supabase: estrategia de versionado, actualización segura,
automatización con Dependabot y Renovate, mantenimiento del
SDK (Flutter, Node, Supabase CLI), auditoría de seguridad y
migraciones ante deprecación.

ENFOQUE

Versionado y salud de dependencias

STACK

Flutter + Dart + npm + Actions

NIVEL

Básico a Avanzado

TIEMPO ESTIMADO

6-8 horas

Dependabot

Renovate

pub

npm

Flutter

Seguridad

Contenido

7 capítulos · Mantenimiento de dependencias (Flutter + npm + GitHub Actions)

PARTE 1 · ESTRATEGIA Y FLUJO DE ACTUALIZACIÓN

01 **01 — Fundamentos de Dependencias**

02 **02 — Flujo de Actualización**

PARTE 2 · AUTOMATIZACIÓN Y SDK

03 **03 — Automatización: Dependabot y Renovate**

04 **04 — Mantenimiento del SDK y Ecosistema**

PARTE 3 · SEGURIDAD Y CICLO DE VIDA

05 **05 — Seguridad y Auditoría**

06 **06 — Deprecación, EOL y Migraciones**

PARTE 4 · OPERACIÓN CONTINUA

07 **07 — Makefile y CI para Mantenimiento Continuo**

01 — Fundamentos de Dependencias

Cómo declarar dependencias correctamente, entender el lockfile y resolver conflictos.

1. Estrategia de Versionado

1.1 Rangos Caret (^)

El `^` es el más usado en pubspec.yaml:

```
dependencies:  
  flutter_bloc: ^9.1.1    # >=9.1.1 y <10.0.0  
  equatable: ^2.1.0      # >=2.1.0 y <3.0.0
```

Significado: `^versión` equivale a `>=versión <siguiente-versión-mayor`.

Constraint	Rango real
<code>^1.2.3</code>	<code>>=1.2.3 <2.0.0</code>
<code>^0.1.2</code>	<code>>=0.1.2 <0.2.0</code>
<code>^0.0.1</code>	<code>>=0.0.1 <0.0.2</code>

Regla de oro: Usa caret para librerías y paquetes de utility. Permite recibir bugfixes y features menores automáticamente.

1.2 Pinning Exacto

```
dependencies:  
  next: 16.3.1  
  react: 19.2.8
```

Cuándo usarlo:

- Frameworks críticos (Next.js, React)
- Paquetes where un cambio menor puede romper la app
- Dependencias que controlas tú y quieres releases explícitos

Desventaja: No recibes ni siquiera parches de seguridad automáticos. Cada actualización es manual.

1.3 SHA Pinning (GitHub Actions)

```
- uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1
```

Esto es **pinning absoluto**. Ni siquiera la etiqueta `v7` puede cambiar por debajo. Es la práctica más segura para CI/CD.

1.4 Otras Formas de Declarar

```
# Rango abierto (no recomendado)
paquete: any

# Rango exacto
paquete: '>=1.0.0 <2.0.0'

# Sin límite inferior (riesgoso)
paquete: '<3.0.0'

# Git dependency
paquete:
  git:
    url: https://github.com/user/paquete.git
    ref: main

# Path dependency (local)
paquete:
  path: ../mi-paquete
```

Las dependencias Git y path **no deberían llegar a producción** salvo que sea estrictamente necesario.

2. pubspec.lock

2.1 ¿Qué es?

El lockfile **congela** las versiones exactas que resuelve pub después de leer las constraints.

```
# pubspec.lock (fragmento)
packages:
  flutter_bloc:
    dependency: "direct main"
    source: hosted
    version: "9.1.1"
```

2.2 ¿Se comitea?

Tipo de proyecto	¿Lockfile en git?
App	✓ Sí — para que todos los devs usen las mismas versiones exactas
Librería/Paquete	✗ No — el lockfile no se distribuye; cada consumidor resuelve sus propias versiones

2.3 Conflictos en Merge

Cuando dos ramas agregan paquetes diferentes:

```
<<<<<< HEAD
    version: "9.1.1"
=====
    version: "10.0.0"
>>>>>> feature
```

Solución: Ejecuta `dart pub get` después del merge. Dart reconstruirá el lockfile resolviendo ambos conjuntos de dependencias.

3. Resolución de Conflictos

3.1 `dart pub deps`

```
# Árbol completo
dart pub deps

# Solo dependencias directas
dart pub deps --no-dev

# Formato JSON para CI
dart pub deps --json
```

3.2 `dependency_overrides`

Úsalo **temporalmente** para forzar una versión cuando dos paquetes requieren versiones incompatibles:

```
dependency_overrides:
  collection: ^1.18.0
```

Esto es un parche temporal. El objetivo es que el mantenedor del paquete upstream actualice su constraint.

3.2 pubspec_overrides.yaml

A partir de Dart 3.8, pub admite `pubspec_overrides.yaml` como alternativa para overrides locales:

```
# pubspec_overrides.yaml (NO se comitea)
dependency_overrides:
  collection: ^1.19.0
```

A diferencia de `dependency_overrides` en `pubspec.yaml`, este archivo **no se comitea** (debe estar en `.gitignore`). Es ideal para overrides de desarrollo que no deben afectar al resto del equipo.

3.3 Problema Típico

```
Because every version of flutter_bloc depends on bloc ^9.0.0
and every version of bloc 10.x depends on bloc >=10.0.0,
flutter_bloc is incompatible with bloc ^10.0.0.
```

Diagnóstico:

1. `dart pub deps` muestra el árbol
2. Identifica qué paquete está causando la incompatibilidad
3. Busca si hay una versión más nueva del paquete que resuelva el conflicto
4. Si no, usa `dependency_overrides` o espera a que los mantenedores actualicen

4. Convención del Proyecto Real

El monorepo de referencia usa esta estrategia:

Tipo	Convención	Ejemplo
Librerías Flutter	Caret <code>^</code>	<code>flutter_bloc: ^9.1.1</code>
Frameworks web	Pinning exacto	<code>"next": "16.3.1"</code>
GitHub Actions	SHA pinning	<code>actions/checkout@3d3c42e...</code>
Supabase config	Major version	<code>major_version = 17</code>
Deno Edge Functions	Rango mayor	<code>@supabase/supabase-js@2</code>

5. Ejercicio

Dado este `pubspec.yaml` con conflictos:

```
dependencies:  
  flutter_bloc: ^9.1.1  
  bloc: ^10.0.0 # incompatible con flutter_bloc ^9.1.1
```

Pasos:

1. Ejecuta `dart pub deps` para ver el error
2. Identifica la versión de `bloc` que necesita `flutter_bloc`
3. Decide: ¿bajas `bloc` o esperas a `flutter_bloc` 10.x?
4. Documenta la decisión en un ADR

Resumen

1. **Caret** (`^`) para librerías — flexibilidad controlada
2. **Pinning exacto** para frameworks críticos
3. **SHA** para Actions de CI/CD
4. `pubspec.lock` se comitea en apps
5. `dependency_overrides` solo como parche temporal
6. `dart pub deps` es tu herramienta de diagnóstico



Referencias

- [Dart | Pub dependencies](#) — Sintaxis de versionado y tipos de dependencias
- [Dart | Pub deps](#) — Comando `dart pub deps`
- [Dart | Pub upgrade](#) — Comando `dart pub upgrade` y lockfile
- [GitHub | Security hardening for Actions](#) — SHA pinning en acciones de CI/CD

 **Siguiente:** 02-flujo-actualizacion.md — Cómo actualizar dependencias sin romper el proyecto

02 — Flujo de Actualización

Cómo actualizar dependencias Flutter y npm de forma segura, sin romper el proyecto.

1. `dart pub outdated`

1.1 Leer la Salida

```
dart pub outdated
```

```
Showing outdated packages.  
[*] indicates versions that are not mutually compatible.
```

Package Name	Current	Upgradable	Resolvable	Latest
dio	5.9.0	5.11.0	5.11.0	5.11.0
equatable	2.0.5	2.1.0	2.1.0	2.1.0
flutter_bloc	9.1.0	9.1.1	9.1.1	9.1.1
go_router	17.4.1	17.5.0	17.5.0	17.5.0

Columna	Significado
Current	Lo que tienes instalado
Upgradable	Versión a la que puedes subir con <code>dart pub upgrade</code> (solo patch/minor)
Resolvable	Versión máxima que pub puede resolver sin cambiar <code>pubspec.yaml</code>
Latest	Última versión publicada (puede ser mayor)

1.2 Priorizar Actualizaciones

1. **Patch** (3.0.0 → 3.0.1) — seguras, merge automático
2. **Minor** (3.0.0 → 3.1.0) — revisar changelog
3. **Major** (3.0.0 → 4.0.0) — planificar migración

2. dart pub upgrade

2.1 Upgrade Normal

```
# Actualiza todo a la versión resolvable
dart pub upgrade

# Actualiza un paquete específico
dart pub upgrade flutter_bloc
```

Esto actualiza solo **dentro de la constraint**. Si tienes `^9.1.0`, subirá a `9.x.x` pero nunca a `10.0.0`.

2.2 Upgrade con Major Versions

```
# Activar explícitamente major versions
dart pub upgrade --major-versions

# Para un solo paquete
dart pub upgrade flutter_bloc --major-versions
```

Esto **modifica** `pubspec.yaml` para actualizar la constraint: `^9.1.0` → `^10.0.0`.

 **Precaución:** No ejecutes `--major-versions` en todos los paquetes a la vez. Hazlo de uno en uno, revisando breaking changes.

2.3 Equivalente npm

```
# Mostrar desactualizados
npm outdated

# Actualizar dentro de semver
npm update

# Actualizar a latest (cambia package.json)
npm install next@latest

# Auditoría de seguridad
npm audit
```

3. Proceso Seguro de Upgrade

3.1 Flujo

1. Leer CHANGELOG del paquete

↓

2. Buscar migration guide (GitHub releases / docs)

↓

3. dart pub upgrade <paquete>

↓

4. dart pub outdated (verificar que todo esté bien)

↓

5. dart fix --apply --dry-run
dart fix --apply

↓

6. Ejecutar tests: flutter test -j 1

↓

7. Ejecutar build_runner si aplica

↓

8. Compilar: flutter analyze

↓

9. Commit separado con mensaje semántico

3.2 Ejemplo Real: supabase_flutter

1. Ver versión actual
grep supabase_flutter pubspec.lock

2. Revisar changelog
<https://github.com/supabase/supabase-flutter/releases>

3. Actualizar
dart pub upgrade supabase_flutter

4. Migrar si hay breaking changes
- Revisar cambios en supabase_flutter (ej: nueva inicialización)
dart fix --apply

5. Testear
flutter analyze
flutter test -j 1

6. Commit
git add pubspec.yaml pubspec.lock lib/
git commit -m "fix(deps): update supabase_flutter to 2.17.2"

3.3 Casos Especiales por Paquete

Paquete	Qué revisar post-upgrade
supabase_flutter	Cambios en inicialización, nuevas tablas de auth
flutter_bloc / bloc	Nuevos métodos en BlocObserver, cambios en HydratedBloc

Paquete	Qué revisar post-upgrade
<code>fpdart</code>	Cambios en Either/TaskEither API (Dart 3 pattern matching)
<code>go_router</code>	Cambios en ShellRoute, StatefulShellRoute, redirect
<code>dio</code>	Interceptors, cancelación de requests, multipart
<code>isar_community</code>	Regenerar archivos <code>.g.dart</code> con <code>build_runner</code>

4. `dart fix --apply`

```
# Ver qué cambios haría
dart fix --dry-run

# Aplicar cambios automáticos
dart fix --apply
```

`dart fix` aplica migraciones automáticas del SDK de Dart. Es el **primer paso** después de cualquier upgrade. No cubre migraciones de paquetes de terceros.

5. Build Runner y Code Generation

Después de actualizar paquetes que usan code generation:

```
# Limpiar caché primero
dart run build_runner clean

# Regenerar todo
dart run build_runner build --delete-conflicting-outputs
```

Problema común: `build_runner` queda en versión incompatible. Si falla, actualiza `build_runner` primero.

6. npm: Upgrade en Web

```
# Ver todo
npm outdated

# Actualizar todo
npm update

# Paquete específico
npm install next@latest

# Verificar vulnerabilidades
npm audit

# Fix automático (solo patch)
npm audit fix
```

Convención del monorepo: Next.js y React están exact-pinned. Se actualizan manualmente revisando el migration guide.

7. Ejercicio

Escenario: Acaban de publicar `fpdart 2.0.0` con breaking changes. Tu proyecto usa `^1.2.0`.

1. Revisa el changelog de fpdart en GitHub
 2. Ejecuta `dart pub upgrade fpdart --major-versions`
 3. Corre `dart fix --dry-run` y `dart fix --apply`
 4. Si fallan los tests, identifica qué cambió
 5. Actualiza el código para la nueva API
 6. Commit con mensaje semántico
-

Resumen

1. `dart pub outdated` — diagnóstico, no acción
 2. `dart pub upgrade` — actualiza dentro de constraints
 3. `--major-versions` — solo para un paquete a la vez
 4. **Siempre** leer CHANGELOG antes de actualizar
 5. **Siempre** correr `dart fix --apply` después
 6. **Siempre** tests + analyze antes de commitear
-

Referencias

- Dart | Pub outdated — Comando `dart pub outdated`
 - Dart | Pub upgrade — Comando `dart pub upgrade`
 - Dart | dart fix — Migraciones automáticas de código
 - npm Docs | npm-outdated — Comando `npm outdated`
 - npm Docs | npm-audit — Comando `npm audit`
-

 **Siguiente:** 03-automatizacion-dependabot-renovate.md — Automatizar actualizaciones con Dependabot y Renovate

03 — Automatización: Dependabot y Renovate

Configura bots de actualización como parte del workflow del equipo.

1. Dependabot

1.1 Configuración Básica

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5

  - package-ecosystem: "npm"
    directory: "/apps/web"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5

  - package-ecosystem: "pub"
    directory: "/apps/mobile"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
```

Esta es la configuración real del monorepo de referencia.

1.2 Personalización

```
updates:
  - package-ecosystem: "pub"
    directory: "/apps/mobile"
    schedule:
      interval: "weekly"
      day: "monday"
      time: "09:00"
      timezone: "America/Caracas"
    open-pull-requests-limit: 10
    reviewers:
      - "isaac-urdaneta"
    assignees:
      - "isaac-urdaneta"
    labels:
      - "dependencies"
      - "flutter"
    allow:
      - dependency-type: "direct"
    ignore:
      - dependency-name: "flutter_bloc"
        update-types: ["version-update:semver-major"]
```

1.3 Parámetros Clave

Parámetro	Qué hace	Recomendación
<code>schedule.interval</code>	Frecuencia de chequeo	<code>weekly</code> (evita ruido diario)
<code>open-pull-requests-limit</code>	Máy PRs abiertos simultáneos	<code>5-10</code> (no inundar)
<code>reviewers</code>	Quién revisa los PRs	El team lead o dueño del módulo
<code>allow</code>	Filtrar qué actualizar	<code>direct</code> (solo directas, no transitivas)
<code>ignore</code>	Excluir paquetes específicos	Solo para updates problemáticos conocidos
<code>target-branch</code>	Rama base de los PRs	<code>dev</code> si usas Git Flow

1.4 Grupos (`groups`)

Desde 2023, Dependabot permite agrupar dependencias relacionadas en un **solo PR**:

```

- package-ecosystem: "pub"
  directory: "/apps/mobile"
  schedule:
    interval: "weekly"
  groups:
    bloc-family:
      patterns:
        - "bloc"
        - "flutter_bloc"
        - "bloc_test"
      exclude-patterns:
        - "bloc*"
        - "*,test"
    patch-updates:
      update-types:
        - "patch"
      patterns:
        - "*"

```

Recomendaciones:

- Agrupa **patch updates** en un solo PR (bajo riesgo)
- Agrupa **familias de paquetes** (`bloc`, `go_router`, `supabase`) para revisarlas juntas
- **Nunca agrupes major updates** — cada uno merece su propia migración

1.5 Caso Real: Monitorear root `package.json`

El monorepo actual no cubre el `package.json` raíz (commitlint, husky, commitizen):

```

- package-ecosystem: "npm"
  directory: "/"
  schedule:
    interval: "monthly"
  open-pull-requests-limit: 3

```

Estas dependencias de tooling cambian poco, por eso `monthly` es suficiente.

1.6 Dependabot Security Updates

GitHub habilita automáticamente PRs de seguridad para CVEs conocidas. No necesita configuración en `dependabot.yml`, se activa desde Settings → Code security → Dependabot security updates.

2. Renovate

2.1 ¿Por qué usar Renovate?

Aspecto	Dependabot	Renovate
Configuración	YAML simple, opciones limitadas	JSON/JS, altamente configurable

Aspecto	Dependabot	Renovate
Agrupación de PRs	<code>groups</code> (desde 2023) para version y security updates	<code>groupName</code> para agrupar por tipo
Schedule por paquete	No	Sí, por regex
Auto-merge condicional	Limitado	Sí, con reglas
Dashboard	No	Panel web con estado
Onboarding	No	PR de configuración inicial

2.2 Configuración Mínima

```
// renovate.json
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": [
    "config:recommended"
  ],
  "labels": ["dependencies"],
  "schedule": ["before 9am on monday"],
  "packageRules": [
    {
      "matchPackageNames": ["flutter_bloc", "bloc"],
      "groupName": "bloc packages",
      "automerge": true
    },
    {
      "matchUpdateTypes": ["patch"],
      "automerge": true,
      "automergeType": "pr"
    },
    {
      "matchPackageNames": ["next", "react", "react-dom"],
      "enabled": false
    }
  ]
}
```

2.3 Agrupación Inteligente

```
{
  "packageRules": [
    {
      "matchPackagePatterns": ["*"],
      "groupName": "all dependencies",
      "groupSlug": "all-deps",
      "schedule": ["before 9am on monday"]
    }
  ]
}
```

Cuándo agrupar:

- Paquetes del mismo ecosistema (`bloc` , `flutter_bloc` , `bloc_test`)
- Patch updates (bajo riesgo)
- Dev dependencies

Cuándo NO agrupar:

- Major updates (riesgo alto)
 - Paquetes con migraciones complejas
-

3. Flujo de Trabajo con PRs de Dependencias

3.1 Revisión

1. Dependabot/Renovate abre PR
↓
2. CI corre automáticamente (tests + analyze)
↓
3. Revisar: ¿CI pasa? ¿Hay breaking changes?
↓
4. Si CI pasa + es patch/minor → merge aprobado
↓
5. Si es major → revisar changelog manualmente
↓
6. Merge con squash + mensaje semántico

3.2 Estrategia de Auto-Merge

```
# Dependabot no tiene auto-merge nativo. La forma recomendada por GitHub es
# combinar dependabot/fetch-metadata + gh pr merge --auto (el PR espera a que CI pase).

# .github/workflows/auto-merge-deps.yml
name: Auto-merge dependencies
on:
  pull_request:
    types: [opened, synchronize]

permissions:
  pull-requests: write
  contents: write

jobs:
  auto-merge:
    if: github.actor == 'dependabot[bot]' || github.actor == 'renovate[bot]'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Fetch dependabot metadata
        id: meta
        if: github.actor == 'dependabot[bot]'
        uses: dependabot/fetch-metadata@v3
        with:
          compat-lookup: true

      - name: Auto-merge patch & minor updates
        if: steps.meta.outputs.update-type == 'version-update:semver-patch' ||
steps.meta.outputs.update-type == 'version-update:semver-minor'
        run: gh pr merge --auto --squash "$PR_URL"
        env:
          PR_URL: ${ github.event.pull_request.html_url }
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
```

3.3 Mensajes de Commit

```
# Patch
chore(deps): update dependency flutter_bloc to 9.1.1

# Minor
feat(deps): update dependency dio to 5.11.0

# Major
chore(deps): migrate flutter_bloc from 9.x to 10.x
```

4. CI de Auditoría Semanal

Workflow que corre `dart pub outdated` semanal y crea un issue si hay dependencias atrasadas:

```

name: Weekly dependency audit
on:
  schedule:
    - cron: '0 9 * * 1' # Lunes 9am
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: subosito/flutter-action@v3
        with:
          flutter-version: '3.47.0'

      - name: Check outdated
        id: outdated
        run: |
          dart pub outdated --no-dev > /tmp/outdated.txt
          cat /tmp/outdated.txt
          if grep -q "up-to-date" /tmp/outdated.txt; then
            echo "All up to date"
          else
            echo "has_outdated=true" >> $GITHUB_OUTPUT
          fi
        working-directory: apps/mobile

      - name: Create issue
        if: steps.outdated.outputs.has_outdated == 'true'
        uses: actions/github-script@v9
        with:
          script: |
            const fs = require('fs');
            const outdated = fs.readFileSync('/tmp/outdated.txt', 'utf8');
            github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: '📦 Dependencies weekly audit',
              body: '```\n' + outdated + '\n```',
              labels: ['dependencies', 'automated']
            });

```

5. Ejercicio

Configura Dependabot para un monorepo con estas características:

- Flutter en `apps/mobile/` (pub)
- Next.js en `apps/web/` (npm)
- Tooling en raíz `package.json` (npm)
- GitHub Actions en `/` (github-actions)

Incluye:

- Schedule: Flutter lunes, web martes, tooling mensual, Actions semanal
- Límite: 5 PRs para Flutter, 3 para web

- Revisores: `team-flutter` y `team-web`
 - Labels: `dependencies` + `ecosystem:{{name}}`
 - Ignorar major updates de `flutter_bloc` temporalmente
-

Resumen

1. **Dependabot** es simple y funciona — ideal para empezar
 2. **Renovate** es más potente — útil si necesitas control fino
 3. **Auto-merge** solo para patch/minor que pasen CI
 4. **Auditoría semanal** en CI para detectar dependencias atrasadas
 5. **Revisión manual** siempre para major updates
-



Referencias

- [GitHub | Configuring Dependabot version updates](#) — Configuración de `dependabot.yml`
 - [GitHub | Dependabot security updates](#) — Security updates automáticos
 - [Renovate Docs | Configuration options](#) — Opciones de configuración
 - [Renovate Docs | Preset config:recommended](#) — Preset recomendado
-

 **Siguiente:** 04-mantenimiento-sdk-ecosistema.md — Mantener el SDK del proyecto

04 — Mantenimiento del SDK y Ecosistema

Mantén las herramientas base del proyecto sincronizadas: Flutter, Supabase CLI y Node.js.

1. Flutter SDK

1.1 El Problema

```
# Dev 1: Flutter 3.41.0
# Dev 2: Flutter 3.44.0
# CI/CD: Flutter 3.47.0

# Misma base de código, diferentes versiones → resultados diferentes
# "En mi máquina funciona" → versión incorrecta
```

1.2 FVM al Rescate

📖 La instalación y uso de FVM está cubierta en [12-GIT-FLOW/04-fvm-version-management.md](#). Aquí solo se cubre el aspecto de mantenimiento.

```
# apps/mobile/.fvm/flutter_sdk_version
3.47.0
```

Checklist de mantenimiento:

- ✓ Todos los devs usan `fvm flutter` en vez de `flutter` directamente
- ✓ VS Code configurado con `dart.flutterSdkPath: ".fvm/flutter_sdk"`
- ✓ CI/CD hardcodea la versión en el workflow YAML
- ✓ IDE y CLI leen la misma versión

1.3 Cuándo Actualizar Flutter

Señal	Acción
Nueva versión stable	Esperar 1-2 semanas *
Release de Dart con features que necesitas	Actualizar inmediatamente
CVE de seguridad en Flutter/Dart	Actualizar urgente

Señal	Acción
Paquete que requieres necesita SDK más nuevo	Actualizar

* Buena práctica basada en experiencia del ecosistema, no es política oficial de Google.

1.4 Proceso de Upgrade

```
# 1. Verificar compatibilidad sin cambiar
fvm flutter --version
fvm releases                # ver versiones estables disponibles

# 2. Instalar nueva versión
fvm install 3.47.0
fvm use 3.47.0

# 3. Probar localmente
fvm flutter pub get
fvm flutter analyze
fvm flutter test -j 1
fvm dart fix --dry-run

# 4. Actualizar CI/CD
# Cambiar flutter-version: '3.44.0' → '3.47.0' en workflows
# Actualizar GitHub Variable si usas una

# 5. Commit
git add apps/mobile/.fvm/flutter_sdk_version
git commit -m "chore(sdk): bump Flutter from 3.44.0 to 3.47.0"
```

1.5 Centralización de Versiones

El monorepo referencia la versión de Flutter en múltiples lugares:

```
.github/workflows/ci-quality.yml:      flutter-version: '3.47.0'
.github/workflows/flutter-android-release.yml: flutter-version: '3.47.0'
apps/mobile/.fvm/flutter_sdk_version: 3.47.0
.github/variables/FLUTTER_VERSION: 3.47.0
```

Práctica recomendada: Usa GitHub Variables o un archivo `.github/workflows/config/versions.env` para centralizar:

```
# En un workflow
- name: Setup Flutter
  uses: subosito/flutter-action@v3
  with:
    flutter-version: ${ vars.FLUTTER_VERSION }
```

Así cambias la versión en un solo lugar.

2. Supabase CLI

2.1 Versiones en `config.toml`

```
# supabase/config.toml
project_id = "rifa-gestion"
major_version = "17"      # PostgreSQL
deno_version = "2"        # Deno runtime
```

2.2 Verificar Versiones

```
# Versión de la CLI
supabase --version

# Estado del stack local
supabase status

# Migraciones pendientes vs aplicadas
supabase migration list
```

2.3 Actualizar Supabase CLI

```
# Instalar/actualizar CLI
brew upgrade supabase/tap/supabase # macOS
npm install -g supabase             # npm
# o descargar release de GitHub

# Reiniciar stack local después de upgrade
supabase stop
supabase start
```

2.4 Cambiar Versión de PostgreSQL

```
# El cambio de major_version requiere reiniciar el stack
# ¡Cuidado! Puede causar incompatibilidades en migraciones
major_version = "17"
```

Cambiar de una major de PostgreSQL a otra (p. ej. 15 → 17) requiere verificar que todas las extensiones y queries sean compatibles.

3. Node.js

3.1 `.nvmrc`

```
# .nvmrc en apps/web/  
24  
  
# Usar la versión del proyecto  
nvm use  
  
# Saber qué versión tienes  
node --version  
  
# Instalar una versión específica  
nvm install 24  
nvm use 24  
echo "24" > .nvmrc
```

3.2 `engines` en `package.json`

```
// apps/web/package.json  
{  
  "engines": {  
    "node": ">=22.0.0",  
    "npm": ">=10.0.0"  
  }  
}
```

Esto da una advertencia temprana si alguien usa una versión incorrecta:

```
$ npm install  
npm WARN EBADENGINE Unsupported engine {  
  package: 'web',  
  required: { node: '>=22.0.0' },  
  current: { node: '20.0.0' }  
}
```

3.3 Cuándo Actualizar Node

Versión	Estado	Acción
Node 18	EOL (Abril 2025)	Migrar ya
Node 20	EOL (Abril 2026)	Migrar a 22 o 24
Node 22	Maintenance LTS	Mantener, planificar migración
Node 24 (Krypton)	Active LTS (actual)	Usar (recomendado)
Node 26	Current	No recomendado para producción hasta su LTS (oct 2026)

Regla: Usa siempre la versión LTS activa que usa tu CI/CD.

4. Política de Versiones por Ecosistema

Herramienta	Dónde se declara	Cadencia de actualización
Flutter SDK	<code>.fvm/flutter_sdk_version</code>	Cada 2-3 meses (cada release estable)
Dart SDK	Atado a Flutter	Automática con Flutter
Supabase CLI	Sistema / npm global	Mensual
PostgreSQL	<code>supabase/config.toml</code>	Solo si nuevas features lo requieren
Deno runtime	<code>supabase/config.toml</code>	Cada 3-6 meses
Node.js	<code>.nvmrc</code> + <code>engines</code>	Cada ciclo LTS (~12 meses)

5. Ejercicio

Tu equipo trabaja en el monorepo. Flutter acaba de lanzar 3.47.0, Node 24 es ahora Active LTS, y Supabase CLI 2.1x tiene cambios importantes.

1. Actualiza Flutter a 3.47.0 siguiendo el proceso de 1.4
2. Actualiza los 3 workflows que hardcodean la versión
3. Actualiza `.nvmrc` a Node 24
4. Actualiza `engines` en `apps/web/package.json`
5. Actualiza Supabase CLI y verifica `supabase status`
6. Commitea cada cambio por separado con mensajes semánticos

Resumen

1. **FVM** asegura que todos usen la misma versión de Flutter
2. **Centraliza** la versión en GitHub Variables o un archivo `.env`
3. **Espera 1-2 semanas** antes de adoptar un nuevo Flutter stable (buena práctica empírica, no política oficial)
4. **Supabase CLI** se actualiza independientemente del proyecto
5. **Node.js** se declara en `.nvmrc` y `engines`
6. Cada ecosistema tiene su propia cadencia de actualización

Referencias

- [Flutter | Compatibility policy](#) — Política de soporte de versiones
 - [Flutter | SDK archive](#) — Historial de releases estables
 - [FVM](#) — Flutter Version Management
 - [Node.js | package.json engines](#) — Campo `engines` en package.json
-

 **Siguiente:** 05-seguridad-auditoria.md — Auditoría de seguridad en dependencias

05 — Seguridad y Auditoría

Detecta y responde a vulnerabilidades en dependencias de forma sistemática.

1. `dart pub deps` como Herramienta de Auditoría

```
# Árbol completo con versiones exactas
dart pub deps

# Formato JSON para procesar en CI
dart pub deps --json > deps.json

# Solo dependencias principales (sin dev)
dart pub deps --no-dev
```

Con `dart pub deps --json` puedes analizar el árbol con `jq`:

```
# Extraer todas las dependencias directas
cat deps.json | jq '.packages | to_entries[] | select(.value.kind == "direct main") | {name: .key, version: .value.version}'

# Buscar un paquete específico
cat deps.json | jq '.packages | to_entries[] | select(.key == "supabase_flutter")'
```

2. OSV.dev y GitHub Advisory Database

2.1 Consultar Vulnerabilidades Conocidas

```
# Usando OSV.dev CLI (Open Source Vulnerabilities)
pip install osv
osv query --package supabase-flutter

# O usando la API directamente
curl -X POST https://api.osv.dev/v1/query \
  -H "Content-Type: application/json" \
  -d '{"package": {"name": "supabase-flutter", "ecosystem": "Pub"}, "version": "2.17.2"}'
```

2.2 GitHub Advisory Database

- URL: <https://github.com/advisories?query=ecosystem%3Apub>
 - Filtra por ecosistema **Pub** (Dart/Flutter) o **npm**
 - Cada advisory incluye: descripción, severidad (CVSS), versión afectada, versión corregida
-

3. **npm audit**

```
# Auditoría completa
npm audit

# Solo vulnerabilidades altas y críticas
npm audit --audit-level=high

# Fix automático (patch seguro)
npm audit fix

# Fix solo para vulnerabilidades altas
npm audit fix --audit-level=high

# Formato JSON para CI
npm audit --json > audit-report.json
```

3.1 Integrar en CI

```
# Fragmento de nextjs-ci.yml
- name: npm audit
  run: |
    npm audit --audit-level=high
  working-directory: apps/web
  continue-on-error: true # No bloquear el build, pero reportar
```

4. Dependabot Security Updates

A diferencia de los **version updates** (PRs semanales), los **security updates** son PRs automáticos que GitHub crea cuando se detecta una CVE en una dependencia del proyecto.

4.1 Activación

Settings → Code security → Dependabot → Dependabot security updates → Enable

4.2 Flujo

1. GitHub detecta CVE en una dependencia del proyecto

↓

2. Crea PR automático con la versión parcheada

↓

3. CI corre automáticamente

↓

4. Tú revisas y mergeas

↓

5. GitHub marca la vulnerabilidad como resuelta

4.3 Diferencia con Version Updates

Aspecto	Version Updates	Security Updates
Gatillo	Schedule (config en <code>dependabot.yml</code>)	CVE detectada (automático)
Prioridad	Baja	Alta
Auto-creación	Configurable	Siempre
Se puede desactivar	Sí (en <code>dependabot.yml</code>)	Sí (Settings)

5. Severidades CVSS

```
graph LR
  A[CVSS Score] --> B["0.1-3.9: Low"]
  A --> C["4.0-6.9: Medium"]
  A --> D["7.0-8.9: High"]
  A --> E["9.0-10.0: Critical"]
```

5.1 Matriz de Respuesta

Severidad	Acción	Plazo
Critical (9.0-10.0)	Actualizar inmediatamente. Parche manual si no hay fix	< 24h
High (7.0-8.9)	Actualizar en el próximo ciclo de trabajo	< 1 semana
Medium (4.0-6.9)	Planificar en el próximo sprint	< 1 mes
Low (0.1-3.9)	Documentar y revisar en mantenimiento trimestral	< 3 meses

5.2 Sin Parche Disponible

Si hay CVE crítica pero no hay versión parcheada:

1. Evaluar si la vulnerabilidad afecta tu caso de uso

2. Si afecta: buscar paquete alternativo
 3. Si no afecta: documentar la decisión y monitorear
 4. Crear un issue para dar seguimiento
-

6. Estrategia de Seguridad en CI

6.1 Pipeline Ideal

```
name: Security audit
on:
  schedule:
    - cron: '0 6 * * 1' # Cada lunes
  push:
    branches: [main, dev]

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      # Flutter audit
      - uses: subosito/flutter-action@v3
        with:
          flutter-version: '3.47.0'
      - name: Dart deps audit
        run: |
          dart pub deps --json > /tmp/deps.json
          dart pub outdated --no-dev > /tmp/outdated.txt
        working-directory: apps/mobile

      # npm audit
      - name: npm audit
        run: npm audit --audit-level=high
        working-directory: apps/web
        continue-on-error: true

      # Publicar reporte
      - name: Upload audit report
        uses: actions/upload-artifact@v4
        with:
          name: security-audit
          path: |
            /tmp/deps.json
            /tmp/outdated.txt
```

6.2 Gate en PRs

```
# Bloquear PRs si hay CVE conocida en nuevas dependencias
- name: Check for known vulnerabilities
  run: |
    dart pub deps --json | jq -r '
      .packages | to_entries[] |
      "\(.key)@\(.value.version)"
    ' | while read pkg; do
      echo "Checking $pkg..."
      # Consultar OSV.dev API
      name=$(echo $pkg | cut -d@ -f1)
      version=$(echo $pkg | cut -d@ -f2)
      response=$(curl -s -X POST https://api.osv.dev/v1/query \
        -d "{\"package\": {\"name\": \"$name\", \"ecosystem\": \"Pub\"}, \"version\": \"$version\"}")
      if echo "$response" | grep -q '"vulns"'; then
        echo "VULNERABILITY FOUND in $pkg"
        echo "$response"
        exit 1
      fi
    done
  working-directory: apps/mobile
```

7. Ejercicio

1. Ejecuta `dart pub deps --json` en tu proyecto y extrae las dependencias directas con `jq`
2. Consulta el GitHub Advisory Database para `Pub` ecosystem y encuentra una CVE reciente
3. Integra `npm audit --audit-level=high` en el workflow `nextjs-ci.yml` del monorepo
4. Simula un escenario: se descubre CVE crítica en `dio` 5.9.0 — ¿cuál es tu plan de acción?

Resumen

1. `dart pub deps` es tu primera herramienta de auditoría
2. **OSV.dev** y **GitHub Advisory Database** para consultar CVE conocidas
3. `npm audit` para dependencias web
4. **Dependabot Security Updates** para PRs automáticos de seguridad
5. **Matriz de respuesta CVSS**: Critical <24h, High <1sem, Medium <1mes
6. **Gate en CI** para bloquear nuevas dependencias con vulnerabilidades conocidas



Referencias

- **OSV.dev** | **API** — Consulta de vulnerabilidades por paquete y versión
- **GitHub** | **Advisory Database** — Base de datos de CVEs con filtro por ecosistema

- [npm Docs | npm-audit](#) — Comando `npm audit`
 - [FIRST | CVSS v3.1](#) — Especificación del sistema de puntuación CVSS
 - [GitHub | Dependabot security updates](#) — Security updates automáticos
-

 **Siguiente:** 06-deprecacion-eol-migraciones.md — Deprecación y migraciones

06 — Deprecación, EOL y Migraciones

Identifica cuándo un paquete o herramienta está llegando a su fin y planifica la migración.

1. Señales de Deprecación

1.1 Indicadores Tempranos

Señal	Riesgo	Acción
Sin commits en >12 meses *	Alto	Buscar alternativa
Issues sin respuesta del maintainer	Alto	Evaluar fork o alternativa
Discord/reddit recomendando migrar	Medio	Investigar
Package no soporta última versión de Dart	Alto	Migrar urgente
El paquete tiene dependencias obsoletas a su vez	Medio	Monitorear

* Heurística basada en actividad del ecosistema open-source. No es una regla formal: algunos paquetes maduros tienen pocos commits pero siguen siendo funcionales y estables.

1.2 Ejemplos Reales en el Ecosistema Dart/Flutter

Paquete Obsoleto	Reemplazo	Razón
<code>hive</code> / <code>hive_flutter</code>	<code>isar_community</code>	Hive sin mantenimiento activo, Isar es más rápido y moderno
<code>pennylane</code>	<code>fpdart</code>	pennylane no migró a Dart 3; fpdart tiene pattern matching, Either, Option
<code>built_value</code>	<code>freezed</code> / <code>json_serializable</code>	built_value es verboso; freezed es más idiomático en Dart 3
<code>dartson</code>	<code>json_serializable</code>	dartson abandonado desde Dart 2
<code>flutter_auth</code> / <code>firebase_auth</code>	<code>supabase_flutter</code>	Depende del backend; migración a Supabase si aplica

1.3 Cómo Detectar

```
# Última publicación en pub.dev
dart pub outdated | grep <paquete>

# Verificar estado del repo
# 1. Ir a github.com/<user>/<repo>
# 2. Ver pestaña "Insights" → "Contributors"
# 3. Último commit, últimas releases

# Verificar compatibilidad con Dart actual
# pub.dev → Analysis tab
```

2. Proceso de Migración

2.1 Flujo General

1. Investigar alternativa madura
 - Comunidad activa
 - Soportada activamente
 - Compatible con Dart/Flutter actual

↓
2. Probar en rama separada

```
git checkout -b migrate/fpdart-2.0
```

↓
3. Crear capa de compatibilidad temporal
 - Wrapper/facade para aislar el cambio

↓
4. Migrar gradualmente
 - Feature flags si es grande
 - Un módulo a la vez

↓
5. Eliminar dependencia antigua
 - `dart pub remove <paquete>`
 - Revisar imports huérfanos

↓
6. Correr tests completos

↓
7. Actualizar guías del equipo
 - ADR de la decisión
 - Documentar breaking changes

2.2 Ejemplo: Migrar pennylane → fpdart

```
// ANTES: pennylane (Dart 2)
Either<Failure, User> getUser(String id) {
  return Right(user);
}

// DESPUÉS: fpdart (Dart 3)
Either<Failure, User> getUser(String id) {
  return Either.right(user);
}
```

Cambios comunes:

- `Right(value)` → `Either.right(value)`
- `Left(value)` → `Either.left(value)`
- `Option.of(value)` → `Option.fromNullable(value)`
- Pattern matching con Dart 3 (`switch` + `sealed class`)

2.3 Estrategia: Capa de Compatibilidad

Para migraciones grandes, crea un wrapper:

```
// lib/core/compat/either_compat.dart
// Temporal: migrando de pennylane a fpdart
import 'package:fpdart/fpdart.dart' as fpdart;
import 'package:pennylane/pennylane.dart' as penny;

// Re-exportar la nueva implementación
export 'package:fpdart/fpdart.dart';
```

Luego, módulo por módulo, cambias los imports del wrapper al paquete final.

3. EOL de Flutter y Dart

3.1 Política de Soporte de Google

Según la [compatibility policy](#) oficial, solo la última versión estable recibe soporte activo con parches completos:

```
Flutter stable:
  N (latest)      → Soporte activo (parches completos)
  N-1 o anterior → Sin soporte oficial garantizado
```

Ejemplo con Flutter 3.47.0 actual (agosto 2026):

- 3.47.0 → soporte activo
- 3.44.0 → release anterior, sin soporte oficial garantizado

En la práctica, el ecosistema suele mantener N-1 funcional porque los paquetes rara vez exigen la versión más reciente. Sin embargo, **no hay parches de seguridad garantizados** para versiones anteriores a la última estable.

3.2 Calendario de Releases

```
gantt
  title Flutter Release Timeline
  dateFormat YYYY-MM
  section Stable
  Flutter 3.29 :2025-02, 2025-05
  Flutter 3.32 :2025-05, 2025-08
  Flutter 3.35 :2025-08, 2025-11
  Flutter 3.38 :2025-11, 2026-02
  Flutter 3.41 :2026-02, 2026-05
  Flutter 3.44 :2026-05, 2026-08
  Flutter 3.47 :2026-08, 2026-11
```

Google publica ~4 releases estables al año (~3 meses entre releases). La compatibility policy oficial solo garantiza soporte para la versión estable más reciente; el estimado de ~8 meses de soporte activo por versión es una **inferencia basada en el ciclo de releases**, no una política publicada.

3.3 Qué Pasa si No Actualizas

```
# pubspec.yaml
environment:
  sdk: ^3.8.0 # <= Si tu SDK está por debajo del mínimo que requiere un paquete
```

- El paquete deja de recibir updates
- Nuevos paquetes pueden no ser compatibles
- Vulnerabilidades de seguridad no se parchean
- `dart pub get` empieza a fallar con versiones nuevas

3.4 Actualizar Dart SDK

```
environment:
  sdk: ^3.13.0 # <= Actualizar cuando migres Flutter (Dart 3.13 con Flutter 3.47)
```

`dart fix --apply` es particularmente útil después de cambiar la constraint del SDK:

```
# Después de actualizar el SDK constraint
dart fix --dry-run # Ver qué cambios sugiere
dart fix --apply   # Aplicar cambios automáticos
```

4. EOL de Paquetes Específicos del Proyecto

4.1 Monitoreo Proactivo

```
# Script para detectar paquetes sin actualizar en >1 año
dart pub deps --json | jq -r '
  .packages | to_entries[] |
  select(.key as $p | [
    "flutter", "flutter_test", "intl", "collection", "meta",
    "async", "typed_data", "vector_math", "material_color_utilities"
  ] | contains([$p]) | not) |
  "\(.key) \(.value.version)"
' | while read name version; do
  echo "Checking $name $version..."
  # Consultar pub.dev API
  last_updated=$(curl -s "https://pub.dev/api/packages/$name" | jq -r '.lastUpdated')
  echo "  Last updated: $last_updated"
done
```

4.2 Registro de Decisiones (ADR)

Cuando decides adoptar o reemplazar un paquete, documenta:

```
# ADR-007: Migrar de pennylane a fpdart

## Contexto
pennylane no ha recibido actualizaciones en 18 meses.
No soporta Dart 3 features (pattern matching, sealed classes).

## Decisión
Migrar a fpdart 1.x, que tiene comunidad activa,
soporta Dart 3, y tiene mejor integración con bloc.

## Consecuencias
- Wrapper temporal en lib/core/compat/
- Migración módulo por módulo en 2 sprints
- -1 dependencia obsoleta, +1 dependencia activa
```

5. Estrategia de Long Term Support (LTS)

Para proyectos en producción:

Nivel	Práctica
Gold	Flutter stable + 1 release atrás; paquetes actualizados a minor más reciente
Silver	Flutter stable actual; paquetes actualizados a última major
Bronze	Flutter sin actualizar por >6 meses; paquetes con 2+ majors de retraso

El monorepo debería aspirar a **Gold**: Flutter stable, paquetes en minor más reciente, con Dependabot para patch y minor automáticos.

6. Ejercicio

1. Identifica un paquete en tu `pubspec.yaml` que no se haya actualizado en >12 meses
 2. Busca alternativas maduras (pub.dev, GitHub stars, fecha de último commit)
 3. Crea un plan de migración con capa de compatibilidad
 4. Documenta la decisión como ADR
 5. Ejecuta la migración y verifica con `flutter analyze` + `flutter test`
-

Resumen

1. **12 meses sin commits** = bandera roja (heurística, no regla formal)
 2. **Migración gradual**: capa de compatibilidad + feature flags
 3. `dart fix --apply` después de cambiar SDK constraint
 4. **ADR** para documentar decisiones de reemplazo
 5. **Flutter EOL**: ~8 meses de soporte activo (inferencia del ciclo de releases, no política oficial)
 6. **Nivel Gold**: Flutter stable + paquetes al día
-



Referencias

- [Flutter | Compatibility policy](#) — Política oficial de soporte de versiones
 - [Flutter | SDK archive](#) — Historial de releases estables
 - [Dart | dart fix](#) — Migraciones automáticas de código
 - [ADR | Architecture Decision Records](#) — Patrón para documentar decisiones técnicas
-

 **Siguiente:** 07-makefile-ci-objetivos.md — Targets de dependencias en Makefile y CI

07 — Makefile y CI para Mantenimiento Continuo

Extiende el Makefile y los workflows CI para monitorear la salud de las dependencias automáticamente.

1. Targets de Dependencias en el Makefile

📖 La sintaxis básica de Makefile se cubre en 10-MAKEFILE/02-sintaxis-basica.md. Aquí solo se muestran targets específicos de dependencias.

1.1 Targets Existentes

[illegible]

1.2 Nuevos Targets Propuestos

```
# — Dependencies maintenance —————

.PHONY: audit dep-report check-constraints dep-tree deps-security

# Auditoría completa de seguridad
audit: audit-mobile audit-web audit-actions
    @echo "✅ Security audit complete"

audit-mobile:
    @echo "🔍 Auditing Flutter dependencies..."
    @cd apps/mobile && dart pub get && dart pub deps --json > /tmp/deps-mobile.json
    @cd apps/mobile && dart pub outdated --no-dev

audit-web:
    @echo "🔍 Auditing npm dependencies..."
    @cd apps/web && npm audit --audit-level=high

audit-actions:
    @echo "🔍 Checking GitHub Actions versions..."
    @for f in .github/workflows/*.yaml; do \
        echo "Checking $$f..."; \
        grep -n "uses:" $$f | grep -v "@@" && echo " ⚠️ Unpinned action in $$f" || true; \
    done

# Reporte de estado de dependencias en markdown
dep-report:
    @echo "# 📦 Estado de Dependencias\n" > /tmp/dep-report.md
    @echo "## Flutter\n" >> /tmp/dep-report.md
    @cd apps/mobile && dart pub outdated --no-dev >> /tmp/dep-report.md
    @echo "\n## npm\n" >> /tmp/dep-report.md
    @cd apps/web && npm outdated --json 2>/dev/null | jq -r 'to_entries[] | "* \(.key): \(.value.current) → \(.value.latest)"' >> /tmp/dep-report.md 2>/dev/null || echo " All up to date" >> /tmp/dep-report.md
    @echo "\n📄 Report generated: /tmp/dep-report.md"
    @cat /tmp/dep-report.md

# Verificar constraints de pubspec.yaml
check-constraints:
    @echo "🔍 Checking dependency constraints..."
    @cd apps/mobile && \
        ( dart pub deps 2>&1 | grep -q "Conflict" && \
        echo "✖ Dependency conflicts found!" || \
        echo "✅ No dependency conflicts" )
    @cd apps/mobile && \
        ( dart pub outdated 2>&1 | grep -q "overridden" && \
        echo "⚠️ dependency_overrides in use (should be temporary)" || \
        echo "✅ No overrides" )

# Árbol completo de dependencias
dep-tree:
    @cd apps/mobile && dart pub deps

# Escanear dependencias con vulnerabilidades conocidas
deps-security:
    @echo "🔍 Scanning dependencies for known vulnerabilities..."
    @cd apps/mobile && dart pub deps --json | \
        jq -r '.packages | to_entries[] | "\(.key)@\(.value.version)"' | \
        while read pkg; do \
            response=$(curl -s -o /dev/null -w "%{http_code}" \
                "https://api.osv.dev/v1/query" \
                -d '{"package":{\ "name":\ "$$(echo $$pkg | cut -d@ -f1 | sed 's/-/_/g')\ }", \
```

```
    \ "ecosystem\":"Pub\"},"version\":"$(echo $$pkg | cut -d@ -f2)\"); \
    [ "$$response" != "200" ] || echo "⚠️ $$pkg may have vulnerabilities"; \
done
```

2. Uso en el Día a Día

```
# Antes de un release
make audit          # Auditoría completa
make dep-report     # Reporte de estado

# Semanalmente
make check-constraints  # Verificar que no hay conflictos
make outdated          # Ver qué se puede actualizar

# Cuando actualizas Flutter SDK
make deps              # Reinstalar dependencias
make check-constraints
make audit
```

3. Workflows CI de Monitoreo

3.1 Auditoría Semanal Automática

```
# .github/workflows/deps-audit.yml
name: Weekly dependency audit
on:
  schedule:
    - cron: '0 9 * * 1' # Lunes 9am UTC

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Setup Flutter
        uses: subosito/flutter-action@v3
        with:
          flutter-version: '3.47.0'

      - name: Setup Node
        uses: actions/setup-node@v7
        with:
          node-version: '24'
          cache: 'npm'
          cache-dependency-path: apps/web/package-lock.json

      - name: Flutter audit
        id: flutter-audit
        run: |
          cd apps/mobile
          dart pub get
          dart pub outdated --no-dev > /tmp/flutter-outdated.txt
          echo "has_outdated=false" >> $GITHUB_OUTPUT

      - name: npm audit
        id: npm-audit
        run: |
          cd apps/web
          npm audit --json > /tmp/npm-audit.json 2>&1 || true
          echo "audit_done=true" >> $GITHUB_OUTPUT

      - name: Upload audit artifacts
        uses: actions/upload-artifact@v4
        with:
          name: dependency-audit-${{ github.run_id }}
          path: |
            /tmp/flutter-outdated.txt
            /tmp/npm-audit.json
```

3.2 Gate en PRs

```
# Fragmento para agregar a ci-quality.yml
- name: Check dependency health
  run: |
    cd apps/mobile
    dart pub deps --json > /tmp/deps.json

    # Verificar dependency_overrides (deben ser temporales)
    if grep -q "dependency_overrides" pubspec.yaml; then
      echo "⚠ Warning: dependency_overrides found in pubspec.yaml"
    fi

- name: npm audit gate
  run: |
    cd apps/web
    npm audit --audit-level=high || \
      (echo "x High/critical vulnerabilities found"; exit 1)
```

3.3 Issue Automático por Dependencias Atrasadas

```
# Continuación del workflow semanal
- name: Create issue if outdated
  if: steps.flutter-audit.outputs.has_outdated == 'true'
  uses: actions/github-script@v9
  with:
    script: |
      const fs = require('fs');
      const outdated = fs.readFileSync('/tmp/flutter-outdated.txt', 'utf8');

      // Verificar si ya existe un issue abierto similar
      const issues = await github.rest.issues.listForRepo({
        owner: context.repo.owner,
        repo: context.repo.repo,
        state: 'open',
        labels: ['dependencies', 'automated']
      });

      if (issues.data.length === 0) {
        await github.rest.issues.create({
          owner: context.repo.owner,
          repo: context.repo.repo,
          title: '📦 Weekly dependency report',
          body: ``\n' + outdated + '\n``\n_Automated report_',
          labels: ['dependencies', 'automated']
        });
      }
    }
```

4. Checklist de Mantenimiento Regular

Diario / Por PR

- ☐ flutter analyze pasa sin errores

- ☐ CI verde

Semanal (Lunes)

- ☐ `make audit` — revisar dependencias
- ☐ Revisar PRs abiertos de Dependabot/Renovate
- ☐ Merge patch y minor que pasen CI

Mensual

- ☐ `make dep-report` — generar reporte
- ☐ Revisar issues de seguridad pendientes
- ☐ Actualizar `pubspec.lock` y `package-lock.json`

Trimestral

- ☐ Verificar si hay paquetes deprecados (06)
- ☐ Revisar si Flutter tiene nueva versión stable
- ☐ Evaluar migración de SDK si aplica
- ☐ Actualizar documentación del proyecto

Anual

- ☐ Auditoría completa de seguridad
- ☐ Revisar EOL de Node.js
- ☐ Evaluar todas las dependencias contra alternativas
- ☐ Actualizar Flutter SDK si está 2+ releases atrás

5. Ejercicio

1. Agrega los targets `audit`, `dep-report`, `check-constraints` al Makefile del monorepo
2. Crea un workflow semanal de auditoría que corra `make audit`
3. Configura un gate en `ci-quality.yml` que ejecute `npm audit --audit-level=high`
4. Prueba los targets localmente:

```
make audit
make dep-report
make check-constraints
```

Resumen

1. **Makefile targets** extienden los existentes con auditoría y reportes
 2. **CI semanal** corre automáticamente y crea issues si hay problemas
 3. **Gate en PRs** bloquea códigos con dependencias vulnerables
 4. **Checklist** de mantenimiento: diario, semanal, mensual, trimestral y anual
 5. **Madurez**: estos targets convierten el mantenimiento de rutina en algo automático
-



Referencias

- [GitHub | Schedule events in Actions](#) — Disparadores `cron` en workflows
 - [Dart | Pub deps](#) — Comando `dart pub deps`
 - [npm Docs | npm-audit](#) — Comando `npm audit`
-

Fin del módulo 17. Sigue practicando con el checklist de mantenimiento regular. Cada dependencia que actualices te hará más rápido en la próxima.